

AgentFlame: 面向 AI 编程 Agent 系统效应的语义归因剖析

AgentSight 研究原型

2026 年 6 月 19 日

摘要

AI 编程 agent 的可观测性问题不只是“模型调用了哪个 tool”，而是用户语义意图如何导致真实系统副作用：进程、文件、网络、测试、失败重试和 token 消耗。现有 LLM observability 工具擅长单次 trace tree、span duration、tool call 和 cost；传统 profiler 擅长聚合系统行为；二者分开时仍难回答：哪个任务语义造成了哪些重复、沉重或越界的系统足迹？本文提出 AgentFlame，一种语义系统效应剖析模型：本地小模型只给 session、prompt 和 LLM call 生成一个词标签；tool、shell、child process 与 file/network effect 通过确定性 lineage 继承这些标签；最终以 folded stack 聚合为 flamegraph。在 R170 current refresh 的 325 个可读 Codex/Claude 真实 session 上，AgentFlame 标注 2,859 个 prompt rows 和 114,837 个 LLM-call events，0 个最终标签失败；它把 183,714 个系统观测折叠成 26,829 条语义系统栈，并记录 35,136 次真实 llama.cpp tag calls。去掉 session/prompt 语义后，nonsemantic 和 flat effect baselines 分别有 90.402% 和 90.918% 的观测权重落入混合多个语义区域的 buckets；R224 消融显示 prompt-only 将 mixed weight 降到 36.722%，说明 prompt 语义帧能分开传统 process/effect summary 会合并的 task-effect 区域。本文也明确边界：R114/R191/R229/R232/R234 在固定 command-mode、外部仓库和受控 target-network probes 中给出 100.0% scoped precision/recall 且 negative-control joins 为 0；但 Codex/Claude-launched target-network rows 仍有 orphan/missing-action cases。因此用户任务实验、任意仓库泛化、更多 agent family、same-tag duplicate prompt-row identity 和人工标签 adequacy 仍未完成，不能声称用户效用或完整系统 provenance 已被证明。

1 问题

一个 AI 编程 agent 运行结束后，开发者通常不想逐条回放消息。他们想知道：这次运行哪里最重，哪里绕路，哪里反复试错，哪些文件或网络目标被碰过，哪类用户请求导致最多测试、最多读取、最多失败或最多 token。传统工具各自只覆盖一半。LLM trace 工具能展示 agent、LLM call、tool call、prompt、completion、latency 和 cost。进程或文件审计工具能展示 git、cargo、rg、sed、docker 等系统行为。Span-duration flamegraph 甚至已经出现在 multi-agent workflow debugging 中，用于看 critical path、parallelism、error cascade 和 tool overhead。

但是这些视图仍难回答跨层问题：

为了哪个用户任务，agent 反复运行了哪些命令、读取了哪些路径、产生了哪些网络或文件副作用？

本文的核心判断是：agent observability 的对象不应只是 span，也不应只是进程，而应是 *task-grounded system effect*。我们把目标栈写成：

```
sessionTag;promptTag;llmcall/tool;process*;effect
```

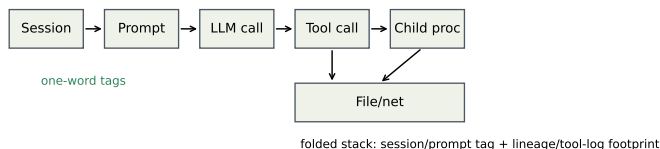


图 1: AgentFlame 的归因模型。小模型只标注 session、prompt、LLM-call 语义；tool/process/file/network effects 通过确定性 lineage 继承语义帧并折叠成 stacks。

这个模型把两类传统工具分开的信息合并起来。上层的 session、prompt、LLM call 用本地小模型压缩成一个词；下层 tool、process、file/network effect 不由模型猜测，而通过确定性关联继承语义上下文。最后用 folded stack 聚合重复路径，让宽度表示 event count、effect weight 或 token weight，而不是只表示 span duration。

本文的贡献不应表述为“agent flamegraph”。已有系统已经把 agent spans 画成 flamegraph。本文的贡献是一个更具体的系统归因模型：在 agent-native local histories 中把用户语义意图与系统行为代理接起来，并为 live AgentSight traces 定义 exact provenance 目标，使跨 session 的重复副作用可聚合、可检查、可比较。

2 设计

图 1 展示了 AgentFlame 的设计。语义层非常小：每个 session、prompt 和 LLM call 只生成一个 lowercase word，例如 refactor、review、test、research。这些词不是固定 ontology，也不是 verdict；它们只是给折叠栈提供短、稳定、可搜索的任务帧。

系统层不交给 LLM 判断。Agent-native 历史模式从 Codex/Claude JSONL 中恢复 tool 类型、命令入口、状态、路径组和粗 effect 类别。AgentSight exact 模式的目标输入是：

```
tool_call -> shell -> child process ->
file/network effect
```

每个 in-scope effect 必须能追溯到 tool call 和 prompt ancestry。如果只能用 PID 或时间戳模糊匹配，就不能作为强 provenance claim。因此，语义由上层继承，系统事实由 instrumentation 或 session parser 产生。

2.1 为什么是一个词

一个词标签有三个工程优点。第一，火焰图帧必须短；长摘要会让图不可读。第二，标签只用于导航和聚合，不承担安全、正确性或因果判决。第三，一个词标签容易本地化和缓存化。当前实现对 session/prompt/LLM 文本各处理一次，后续数十万条系统事件只做普通程序关联和聚合。

这也约束了论文 claim。AgentFlame 不证明标签是真实意图 ontology。它先证明一个更可审计的机制：加入这些短语义帧后，传统 baseline 会合并掉的系统效应是否被分开。

2.2 Stack 语法

系统效应栈形如：

```
project;agent;session;prompt;
call:tool/...;process*;effect;path/domain;status
```

Token 栈形如：

```
project;agent;session;prompt;
call:llm/...;model;kind
```

完整视图保留 session、prompt、LLM-call 语义。消融视图删除部分语义轴：session-system、prompt-system、llm-token 等。所有视图共享同一事实表；投影不应改变总权重。

2.3 可视化模型

AgentFlame 把所有信息塞进一张图。面向开发者的核心界面应有四类互补视图。第一，semantic system flamegraph 用宽度表达 system-effect count，用于回答“哪里重、哪里绕、哪里重复”。它适合跨 session 聚合，因为 folded stack 会把同一语义任务下重复出现的 rg、cargo test、文件读取或网络目标折叠到同一横条。Duration 是单独的 projection：当前 R225 只支持 prompt wall-clock baseline，不支持 active runtime 或 tool/LLM span-duration。第二，token flamegraph 使用同样的 session/prompt/LLM-call 语义帧，但叶子变成 model/prompt/completion token，回答“哪类任务消耗上下文和模型预算”。

第三，exact-lineage tree 不是聚合图，而是 provenance drill-down。它展示一个 prompt 下的 tool call、shell、child process 和 file/network effect 链，用来确认某个横条是否真的来自目标 agent process family。R182/R183 说明这个视图不能省略：低层 codex network rows 可以被 join，但只有 R191 这种 target python3 child-process rows 也被观察并 join 后，UI 才能把对应固定 workload 标成 covered。第四，claim/evidence board 把每个 claim 连接到 artifact、oracle 和 status，防止用户把 smoke run 当成完整结论。

因此 flamegraph 与 tree 的职责不同。Flamegraph 负责发现热点和重复模式；tree 负责解释一个热点能不能被信任；claim board 负责告诉用户哪些结论还只是 partial。这种三层设计是传统 profiler 和普通 agent trace 都缺的：前者没有 prompt 语义，后者通常没有系统副作用的 deterministic ancestry，更不会把 claim boundary 作为一等视图。

3 实现

本文的实验结果来自 AgentFlame 研究原型及其生成 artifacts。该原型当时是独立 Rust CLI，并输出 agentflame.json、tags.json、folded stack 文件、SVG 和 HTML dashboard。当前开源仓库的产品化入口已经收敛到 agentpprof/：它复用 agent-session，输出 pprof-compatible .pb.gz、folded、SVG 和 JSON。因此，本文的 headline 数字评估的是 AgentFlame 研究原型；agentpprof 是产品化方向，除 smoke artifacts 外还没有替代本文所有 RQ1-RQ6 结果。默认输出路径是：

```
.agentsight/agentflame/latest/
```

本次 full run 的命令为：

```
cargo run --manifest-path
agentflame/Cargo.toml -- run --project-root
```

```
. --scan-files 10000 --max-sessions
10000 --llama-url http://127.0.0.1:18080
--model local-r170 --timeout 60 --out
.agentsight/agentflame/r170-full-current
```

这条命令是研究 artifact 的 provenance，不是当前仓库的用户入口。R170 的 committed summary 记录 repo_dirty=true，因此本文把它作为 dirty-provenance mechanism evidence 使用：数值可以用于当前本机历史 characterization 与 folded-total 检查，但它不是 clean release artifact run，也不支持 C5/C6 outcome claims。当前用户入口示例是：

```
cargo run --manifest-path
agentpprof/Cargo.toml -- --project-root .
-o agent.pb.gz
```

模型为本地 qwen2.5-3b-instruct-q4_k_m.gguf，通过 llama.cpp server 运行，temperature 为 0，并用 grammar 约束输出一个词。AgentFlame 没有 heuristic fallback：如果 LLM server 缺失，或模型重试后仍不能输出合法一词标签，run 失败。

3.1 隐私与可复现

原始 agent trace 不提交。agentflame.json 默认包含 redacted previews、hash、tag、计数和路径组；tags.json 保存 tag cache 和 LLM provenance，不保存原始 prompt 文本。本次 run 遇到一个 root-owned Claude JSONL，不可读；系统没有要求 root 权限读取，而是跳过并写入 warnings。这让覆盖范围可审计，而不是静默假装完整。

3.2 与 AgentSight 的关系

AgentFlame 的第一模式是零 instrumentation 的历史分析。AgentSight 的独特点是运行时 exact provenance：tool_call -> shell -> child process -> file/network effect。正确集成后，AgentFlame 负责语义帧，AgentSight 负责系统效应事实，两者通过 tool/session/prompt ancestry 连接。当前 full run 还不是 live exact-effect 结果；它验证的是 agent-native history projection。Live lineage 证据来自独立 scoped suites：R114/R191/R229/R232/R234 覆盖固定 Codex command-mode、target HTTP probe、controlled multi-workspace、外部仓库和局部 Claude/Codex HTTP-family workloads；R238/R240 进一步修复 direct process/network readiness 并加入 over-attribution regression guard。这些证据支持固定范围内的 exact provenance，但不支持广义 Claude-launched、raw-socket、任意仓库或 HTTP payload/URL reconstruction。R230-R233 则审计 generated full-history projection：folded rows 有 semantic/call/effect frames，display projection 与 event-local prompt tags 一致，duplicate prompt indexes 被降级为 non-keyed row identity。

4 评估

4.1 RQ1：本地小模型能否全量标注？

表 1 给出 R170 current full-history refresh 结果。AgentFlame 分析了 325 个可读 session (198 个 Codex、50 个 Claude、77 个 Claude subagent)，处理 2,859 个 prompt rows 和 114,837 个 LLM-call events。标签器共有 118,021 个请求，82,886 个命中 cache，35,136 次真实 llama.cpp HTTP calls，

表 1: R170 current full-history refresh 的本机 AgentSight session 运行指标。

指标	数值
Readable sessions	325
Codex / Claude / Claude-subagent	198 / 50 / 77
Raw tool events	142,468
Raw LLM events	114,837
Prompt rows	2,859
Unique prompt tags	328
LLM-call tags	114,837
Unique LLM-call tags	1,423
Tag requests	118,021
Cache hits	82,886
llama.cpp HTTP calls	35,136
Successful final uncached tags	35,135
Final tag failures	0

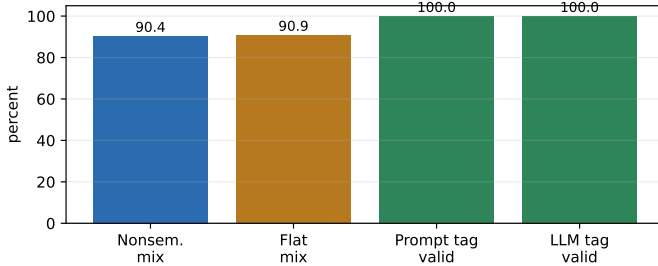


图 2: 当前机制结果。图由 docs/visexp/paper/make_figures.py 从 .agentsight/agentflame/r170-full-current/agentflame.json 重新生成; 它反映 Rust full-run 输出, 而不是旧的 Python sampled-run artifact。

35,135 个成功最终 uncached tags, 0 个最终失败; Prompt 和 LLM-call tag 非法计数均为 0。这说明一词语法和本地 3B 模型路径在真实历史规模上可运行, 但仍不是 C5/C6 outcome evidence。R180 在同一组 300 个 redacted fragments 上进一步确认 0.6B、1.1B 和 3B GGUF 都能输出符合语法的一词标签; 该实验只支持 syntax/latency/stability, 不支持 semantic adequacy。

这个结果不等于标签语义充分。例如某些 tag 明显带噪声: agentsightsm、testcodex、bashoutput。因此系统区分 raw tag 和 canonical display tag: R189 保留 raw tag, 并把 prompt-effect/prompt-row/LLM-event tag 从 263/328/1,423 降到 216/279/1,254, 同时保持 folded 总权重不变, system/token stacks 从 26,829/8,569 降到 26,067/7,661。这只是可审计的候选长尾降噪空间, 不证明 raw tags 一定语义冗余。AgentFlame 因此使用 semantic compaction lifecycle: raw tag 不变, canonical map 只作版本化展示层, pending actions 保存 keep/merge/regenerate/split, regenerated_tag 只有经过 R203 paired/adjudicated promotion review 和 display-map diff 后才能进入 canonical view。R209 将这个 lifecycle 落成可消费数据契约: active display map 每个 raw tag 一行, drilldown index 保留 raw 支持和 top system profile, reviewed diff 只在人工 promotion 后产生。因此 RQ1 支持的是 feasibility 和 syntax claim; tag adequacy 需要单独实验。

4.2 RQ2: projection 方法如何取舍?

R170 full run 产生 183,714 个 system observations, 并折叠成 26,829 条 unique semantic system stacks, 压缩率为 6.848 倍。关键不是压缩本身, 而是 baseline mixing。当删

表 2: 语义栈与 baseline mixing。

指标	数值
System observations	183,714
Unique semantic system stacks	26,829
Semantic compression	6.848x
Max stack reuse	6,004
Nonsemantic mixed buckets	4,685
Nonsemantic mixed weight	90.402%
Flat mixed weight	90.918%

表 3: R224/R170 system-effect 语义轴消融。所有行都保持 183,714 总权重。

Variant	Unique stacks	Bucket	Residual
No semantic	11,967	90.402%	44.716%
Session only	15,027	84.407%	33.434%
Prompt only	24,703	36.722%	7.485%
Session + prompt	26,829	0.000%	0.000%

除 session/prompt 语义轴后, nonsemantic folded baseline 有 4,685 个 mixed buckets, 覆盖 166,081 个观测, 也就是 90.402% 的系统权重。如果进一步退化成 flat effect baseline, mixed weight 占 90.918%。

这为回应“传统工具也能看出来”的质疑提供了机制性证据。传统 flat summary 可以告诉我们 sed read 有 25,755 次、rg read 有 15,336 次、cargo test 有 2,903 次。但是它不知道这些行为分别来自 refactor、review、research、design 还是 test prompt 区域。R211 把这个失败模式量化: rg/sed/git/cargo 分别覆盖 176/180/147/68 个 prompt tags; process:git;effect:read;status:ok 有 116 个 prompt tags, top prompt 只占 24.977%; process:cargo;effect:test;status:ok 有 48 个 prompt tags, non-top-prompt weight 为 68.05%。因此传统 process/effect summary 不是少一个漂亮标签, 而是把许多不同用户意图造成的相同行为合成同一个 bucket。

R251 检查这些 prompt tags 是否只是随机词。它在每个 session 内随机打乱 prompt tags 1,000 次; 真实 tags 对 sanitized process/effect/status behavior 的 session-beyond 信息增益为 8.419%, 高于 null p95 的 1.903% ($p = 0.0010$), top-behavior purity 为 20.196%, 也高于 null p95 的 18.367%。这个结果只支持 tags 与系统行为有 weighted association, 不支持人工语义 adequacy; 低绝对 purity 和 broad tags 仍需要 R124 人工标签。

R224/R131 则把问题改成 projection ablation: 同一份 folded input、同一 183,714 denominator、总权重不变, 只删除或保留 session/prompt 语义轴。表 3 显示 system-effect 分割主要来自 prompt 轴: session-only 只把 mixed full-semantic bucket weight 从 90.402% 降到 84.407%, prompt-only 则降到 36.722%; 按 residual mixed weight 计算, prompt-only 从 no-semantic 的 44.716% 降到 7.485%。完整 session+prompt 的 0.000% mixed weight 是定义上的上界, 只能作为 audit/drilldown 视图, 不能被解释成用户效用。

R223 因此给出的不是单一最佳火焰图, 而是分层默认。No-semantic 适合作为 hotspot baseline: 11,967 个 stacks、15.352x 压缩, 但 90.402% effect weight 被混合。Prompt-only 是 system-effect profile 的最佳单语义轴: mixed/residual mixed weight 降到 36.722%/7.485%, 仍保持 7.437x 压缩。完整 session+prompt 保留给 audit 和 drill-down。Display policy 同样可插拔: R209/R212 的 conserva-

tive display 与 alias-only 等价, 26,612 个 stacks 且 0.0% un-reviewed active weight; profile-guarded candidate view 可降到 26,067 个 stacks, 但会激活 2.532% 未审查 effect weight, 所以只能作为上界消融。Vocabulary overlay 把 raw unique tag strings 从 1,546 降到 1,364, top-20 support coverage 从 93.683% 提升到 95.186%, 但这只是 display-support concentration, 不是 tag correctness。

R225 给出另一个边界: prompt wall-clock duration 与 system-effect count 相关但回答不同问题。它覆盖 324/325 个有 prompt spans 的 sessions 和 183,714/183,714 个 system-effect observations, 总 duration 为 859.019 小时; duration top-10 与 effect-count top-10 只重合 7 个, top-20 重合 12 个, rank Spearman 为 0.623。例如 **network** 在 duration 中排第 8、占 2.837%, 但在 effect weight 中排第 93、占 0.040%; **benchmark** 则相反。历史 artifact 没有 tool/LLM start-end span, 所以 R225 不是 active runtime 或 workflow span-duration trace, 也不能支持 C5。这就是系统设计需要可插拔 projection 的原因: 不同视图优化不同问题, 而不是让小模型一次性决定最终语义真相。

4.3 Case Study: 能看出什么

本次 full run 中, 最大的 semantic stack 是:

```
project:agentsight;agent:codex;session:refactor;prompt:refactor;
call:tool/tool;effect:process;status:ok
```

权重为 6,004。另一个明确的系统例子是 cargo test。Flat summary 只会显示 2,903 次 successful cargo test。Nonsemantic stack 会保留 call:tool/shell;process:cargo;effect:test;status:ok, 但仍把不同语义区域混在一起。Semantic stack 将它拆成多个区域, 包括 review/review、refactor/refactor、research/explain、research/refactor、design/review 等。R211 的 R170 刷新版本进一步显示 process:cargo;effect:test;status:ok 在 48 个 prompt tags 中分布, refactor 只占 31.95%, 其余 68.05% 来自 review、research、commitlog、explain、test 等 prompt。这生成了一个可检查的候选分解: 同样是 test process, 语义来源不同; 后续 C5 才能测试开发者是否能更快或更准确地使用它。

这类结果生成可供检查的候选分解, 用来辅助三类 developer questions:

- 哪个任务导致最多重复测试或读取?
- 哪些 prompt tag 反复触碰同一组路径或域名?
- 哪些失败行为集中在特定 semantic region, 而不是全局系统问题?

Trace tree 能回放某次调用, span flamegraph 能看 duration bottleneck, flat summary 能看重命令。但它们都不直接给出跨 session 的 task-effect 聚合。

4.4 RQ3: exact lineage 是否成立?

当前答案是: 固定 command-mode suite 已经成立, 但 broad full-history provenance 仍不足。R110–R113 先把 agent-run envelope 从 fixture、SQLite export 推到 record --<command> capture path; R113-live 在 5 个只读 codex exec 任务上创建 5/5 sessions/tools 并 join 508/508 raw effects。R114 是主要支持性证据: 20 个固定 Codex tasks 加 per-task negative controls 后, 1,273/1,273 in-scope effects

joined, 0 false positives, 0 false negatives, 3,170 个 negative-control effects 中 0 个被 join; raw join 只有 22.055%, 因为 wrapper、sibling 和 out-of-scope effects 应保持 orphan。

Network 和泛化证据是 scoped 的。R182 修复 record-mode --trace-net 后能 join 35/35 个低层 codex network rows, 但 target rows 仍为 0/0; R191 的固定 Python HTTP probe 才给出 4/4 target rows joined、0/310 negative controls。R229/R232/R234 分别在 controlled multi-workspace、外部 fresh repo 和局部 Claude/Codex HTTP-family workload 上复用同一 oracle, 得到 394/394、353/353+4/4、269+8/8 in-scope/target rows joined, negative controls 均为 0。R235–R240 则故意触碰 raw/multiprocess/Claude-launched boundary: direct controls 被修复并有 regression guard, 但 broader agent-launched target-network coverage 仍 partial。

Full-history projection 是另一类证据。R230 证明 183,714/183,714 folded system-observation weight 都有 semantic session/prompt tag frames 和 call/effect frames; R231 证明 display projection 与 event-local prompt tags 一致; R233 将 duplicate prompt indexes 降级为 non-keyed row identity 后把 normalized semantic drift 降为 0。因此 C4 支持“固定 command-mode suite、controlled external cross-repo workload, 以及局部 Claude/Codex target-network probes 中 exact semantic-effect lineage 成立”, 不支持任意历史、任意仓库、任意 agent、HTTP URL 语义恢复或完整 row-identity proof。

4.5 RQ4: 用户效用是否成立?

当前答案是不成立, 或者更准确地说: 未验证。已有 task packet、answer key、scorer 和 launch package, 但没有真实参与者响应。R184/R186 把状态固定为 ready_for_participant_collection, 并要求先跑真实 R142 五人 pilot。R187/R193/R195/R207 准备了 R142 participant packets、空白 response CSV、R124/R190/R203 label sheets、R195 ingestion/scoring contract 和 return filenames; 这些只支持 launch readiness, 仍记录 0 个参与者响应、0 个人工标签。R242–R247 用 synthetic returns、static forms、headless Chrome 和离线 tarball 验证 collection pipeline 能加载、导出和拒绝错误输入; R244 synthetic exports 不进入 R195 inbox。R249/R255/R258/R259 把 C5 扩到 paper-scale: R258 生成 43-member tarball; R259 生成 12 个 participant forms、6 个 labeler forms、168 行 synthetic C5 merged CSV 和 1,002 行 C6 synthetic labeler rows; R259 仍记录 0 个真实响应和 0 个人工标签, C5/C6/weak-accept gates 保持 false。当前 baseline 应设为 trace tree、event-count-proxy、flat process summary、nonsemantic folded stack 和 semantic folded stack, 比较 accuracy、time、false positives 和 confidence。当前 R142 packet 中原来的 span-like 条件已经明确命名为 event-count proxy, 因为 folded artifact 没有真实 duration; 它不能直接当作 span-duration baseline。R225 已经从 R170 timestamps 重构出 prompt wall-clock duration baseline, 并显示它与 effect-count profile 的 top tags 和排序有明显差异; 但该 baseline 可能包含 idle/user-wait time, 且 tool/LLM 调用仍没有 start-end span, 因此它只能作为下一版 preregistered packet 的 prompt-span baseline, 而不是 active runtime 或完整 workflow span-duration baseline。如果这个实验失败, AgentFlame 仍可能是专家 forensic 工具, 但不能写成提升普通开发者效率。

4.6 RQ5: 小模型成本和标签质量

R123/R180 说明本地小模型路径在工程上可用, 但这只是 syntax、latency 和 repeated-input stability。R123 对 300 个 redacted session/prompt/LLM-call fragments 做三次 identical repeats, 900/900 输出合法, 285/300 fragments 完全稳定, p50/p95 latency 为 11/31 ms。R180 用同一输入比较 0.6b、TinyLlama 1.1b 和 3b GGUF, 三者都是 900/900 valid; exact-stable fragments 为 299/300、279/300、285/300, p95 为 23/18/32 ms。但 TinyLlama 虽然稳定, 却大量输出 `localization/localized`, 说明语法稳定不能替代标签 adequacy。标签问题的核心不是“能不能输出一个词”, 而是如何在不丢 raw drilldown 的前提下减少无意义长尾。R189/R190/R196/R201/R202/R203/R205/R209/R212/R213/R214 形成一个可审计 display-governance pipeline: raw tags 不被覆盖; dictionary aliases 可以 active; lexical/profile merges 和 regenerated labels 在没有人工审核前只能 pending; 系统不使用隐藏 other。当前 display map 中 1,811/1,811 raw rows 都有 active display rows, active display labels 为 1,509, 63 个 deterministic alias rows active, 209 个 candidate rows 和 323 个 review-required rows 只作为 overlay; 482,398 support 全部 preserved, drilldown CSV 保留每个 display bucket 的 raw membership。在 compactness 上, canonical display 把 raw unique tag strings 从 1,546 降到 1,364, top-20 support coverage 从 93.683% 提升到 95.186%; 在 projection 上, alias-only/R209 conservative display 把 system stacks 从 26,829 降到 26,612, 而假设激活所有 profile-guarded candidates 可降到 26,067, 但会激活 2.532% 未审查 effect weight。这套治理机制也有明确失败 gate。R190-score 仍是 `human_labels_empty`, R203 final labels 为 0, canonical map 未更新。R214 还显示 prompt-level review-required support 为 3.258%, 超过 3% 预算; higher-tail-threshold 下 head stability 只有 65.217%, 低于 80% gate。因此系统可以建议合并或重生成, 但不能为了让火焰图更干净就自动提升 candidate merges。前端与 collection runs 只证明这个 contract 可被消费。R215/R216/R217 分别覆盖 TypeScript renderer model、headless browser DOM harness 和 Next production default render: raw/display/pending 都保持 482,398 support, display/pending buckets 为 1,748, candidate promotion 和错误 drilldown 负例会被拒绝。R218 只用 synthetic review fixtures 验证 promotion gate 会拒绝 unsafe rows, 不证明 promotion quality。R219/R242–R247 把 C5/C6 collection、forms、CSV export 和 scorer handoff 固定下来, 但仍记录 C5 responses 为 0、C6 final adequacy labels 为 0、`weak_accept_supported=false`。OSDI 版本还需要真实 R124/R190/R203 人工标签; 一词标签应被定位为 lossy navigation frames, 而不是 semantic truth。

4.7 RQ6: 开源 artifact 路径是否可用?

R160/R200/R220/R248/R253/R254/R256 检查的是 artifact usability, 不是 C5 用户效用或 C6 标签正确性。R160 用 8 个固定历史 Codex session 文件跑 Rust CLI: clean run 生成 dashboard、folded stacks、SVGs 和 `tags.json`, 76 个 tag requests 中 60 次真实 llama.cpp calls、耗时 1.64 s; cached rerun 76/76 cache hits、0 次 LLM call、0.11 s。R200 换成 public-safe Codex fixture, 并确认没有读取真实 `.codex/.claude` traces、没有 prompt preview 泄漏。R220/R248/R253/R254 把入口推进到产品化 `agentpprof`: clean clone、本地 install、GitHub branch install 和 pinned-

revision install 都能在 committed public fixture 上生成 pprof/folded/JSON/SVG outputs, 且 `go tool pprof -top` 读回 6/6 task samples。R256 验证 crate package 边界: 8 个 intended package files、35,438-byte crate archive、archive/list equality、无 target/docs/out/private-history 泄漏, 并观测到 registry dependency `agent-session v0.3.3`。这些结果说明固定输入下的 local artifact path、public fixture path、install/readback path 和 crate-package dry-run 可审计。它们不等于 crates.io publish/readback、external-machine install、真实 report public sanitization 或社区开发者已经能顺利使用。36-session discovery run 首次标注耗时 173.05 s, cached rerun 因活跃 session 增长又出现 34 次新 LLM call; 因此默认体验仍需要明确采样、缓存和活跃 session 漂移。

4.8 评估完整性审计

表 4 把当前证据按 claim gate 汇总。这个表的作用不是装饰, 而是防止论文把 smoke evidence 写成系统结论。OSDI reviewer 最可能挑战 C4、C5 和 C6: C4 仍有 Codex/Claude-launched target-network orphan/missing-action cases, C5 没有 participant responses, C6 没有人工 adequacy labels。R184/R186/R187/R188/R245/R246 的作用是防止过度声明: 它们把状态固定为 `not_weak_accept`, 区分 launch material 与 outcome evidence, 并确认当前文本没有把缺口写成 supported claims。因此当前最诚实的定位是“supported mechanism plus partial systems artifact”。

4.9 Gate Lesson: 为什么保留 orphan

R183 的作用是防止把“有网络事件”写成“目标 workload 的网络事件已被 lineage 捕获”。R182 能 join 低层 codex network rows, 但 target-specific loopback/child-process rows 为 0/0; 因此后续 gate 要求 target rows 被观察并 join, negative controls 仍保持 orphan。R191/R229/R232/R234 用同一 oracle 扩展到 target HTTP probe、controlled multi-workspace、外部仓库和局部 agent-family workload; R235–R240 则暴露并部分修复 raw/Claude-launched target-network 边界, 同时把 command-root over-attribution 和 target-child capture 写成 regression tests。R230–R233 把同一思想用于 generated full-history projection: folded rows 必须带 semantic/call/effect frames, duplicate prompt indexes 不能伪装成可证明 row identity。这体现本文的系统性: AgentFlame 不把 trace 与 process summary 并排展示, 而是把 claim 也放入 lineage model 下审计, 区分用户任务 effect、agent runtime 噪声和必须保持 orphan 的事件。

5 相关工作

Flamegraphs 与 profiling. Brendan Gregg 的 flamegraph 让用户通过宽度定位 hot stacks。Grafana/Pyroscope、Datadog、Sentry 等系统已经把 flamegraph 用于 CPU、内存、profile 和 traces。AgentFlame 借用 folded stack 表示, 但 stack frame 来自 agent semantic context 与 system-effect lineage, 而不是函数调用栈。

Distributed tracing 与 agent observability. OpenTelemetry/Dapper 风格 tracing 把一次请求表示为 span tree。LangSmith、Langfuse、Phoenix、AgentOps 等工具已经记录 agent、LLM、tool、retrieval、cost 和 latency。SigNoz/Inkeep 公开展示了 multi-agent workflow 的 span-duration flamegraph。这些系统擅长解释单次 workflow 如

表 4: 当前 claim gate 的主文摘要。完整证据 ledger 保存在 CLAIM_VERDICT.md; 这里仅保留 reviewer 需要的 status、最强证据锚点和下一步 gate。

Claim	Verdict	Evidence anchor	Next gate
C1 folded aggregation	supported	R170 folds 183,714 system observations into 26,829 semantic stacks with folded totals matching the report.	Fresh-clone release workflow and public packaging.
C2 one-word tags	syntax supported; adequacy open	R170 tags 2,859 prompt rows and 114,837 LLM-call events with 0 final tag failures; R180 shows local 0.6B/1B/3B-class syntax/latency stability.	Human adequacy labels, not grammar validity.
C3 semantic partitioning	mechanism supported	R170/R224 show nonsemantic/flat mixed weight 90.402%/90.918%; prompt-only reduces system bucket/residual mixed weight to 36.722%/7.485%; R209/R212 keep display compaction reversible.	More repositories, user tasks, token/display ablations, and human-reviewed merge/promotion quality.
C4 exact lineage	fixed/controlled workloads supported; broad scope partial	R114/R191/R229/R232/R234 give scoped precision/recall evidence with zero negative joins; R238/R240 add direct-process readiness and regression guards while exposing Codex/Claude-launched target-network orphan cases.	Agent-launched network boundary, same-tag prompt-row identity, more agent families, and richer target network workloads.
C5 user utility	unsupported	Outcome evidence: none yet. Protocol/logistics readiness includes R142/R187/R207/R249/R255/R258 and the R259 paper-scale static collection kit; there are still 0 participant responses.	Score real paper-scale participant responses under the frozen preregistration.
C6 tag adequacy	syntax/protocol partial; adequacy unsupported	Outcome evidence: no human adequacy labels yet. R180/R251 provide syntax/stability and behavior-association evidence; R252 plus R258/R259 static forms only prepare the C6 label handoff.	R124/R190/R203 independent human labels and adjudication.
C7 artifact usability	partial	R160/R200/R220/R248/R253/R254/R256 cover bounded local artifacts, public fixture, local install, GitHub install, pinned revision, pprof readback, and crate-package dry run.	crates.io publish/readback, external-machine install, sanitized real report, setup docs, write-set audit, and external developer feedback.

何执行; AgentFlame 的目标是跨 session 聚合“哪个语义任务造成哪些系统副作用”。

Token/context profiling. Context 和 token profiling 工具关注 prompt/context window 中的热点。AgentFlame 包含 token flamegraph, 但当前只作为 source-local accounting。论文主贡献是 token/semantic/tool/process/effect 共享同一 folded-stack 语法, 而不是 token cost 本身。

6 局限与下一步

当前版本仍不到 OSDI weak accept。Supported 的部分是 scoped mechanism: R114/R191/R229/R232/R234 给出固定 command-mode、target HTTP probe、controlled multi-workspace、外部仓库和局部 agent-family workload 的 exact-lineage 证据; R230–R233 给出 generated full-history projection/normalization audit; R160/R200/R220/R248/R253/R254/R256 给出本机 artifact、GitHub install、pinned revision、pprof readback 和 crate-package dry-run 证据。Partial 的部分是 capture boundary。R235–R240 修复 direct multi-process readiness race, 并把 command-root fallback 与 target-child network capture 写成 regression guard; 但 R238 仍暴露 Codex/Claude-launched target-network orphan/missing-action cases。同样, R230–R233 解决 semantic drift, 却把 same-tag duplicate row identity 明确降级为尚未证明的问题。Unsupported 的部分是 outcome。C5 仍然没有真实 participant responses: R142/R187/R207/R249/R255/R258/R259 只说明任务包、answer key、scorer、paper-scale bundle 和 static collection forms 已准备好。R259 static collection forms 仍需要真实参与者响应, 不能作为 C5 outcome evidence。C6 仍然没有真实 human adequacy labels: R252/R258/R259 只是把 R124/R190/R203 labeler packets 和 handoff 固定下来; R251 的行为关联、R180 的语法稳定、R189–R218 的 display governance 都不能替代人工 adequacy、merge quality、promotion quality、visual drilldown 或 user utility。Artifact 也还不是社区级结论。仍缺 crates.io publish/readback、

external-machine install、真实 report public sanitization、llama.cpp tagger setup、默认采样策略、full write-set audit 和外部开发者反馈。R245/R246/R250/R260/R261/R262 这类检查只保证文本、provenance、review 或 layout hygiene, 不提升 C5/C6 或 weak-accept gate。

7 结论

AgentFlame 的核心不是再画一张 agent flamegraph, 而是把用户语义意图、LLM/tool 历史和系统副作用放入同一个可折叠、可聚合、可审计的栈模型。当前 full run 说明该模型在真实本地 agent 历史上可运行, 并且语义帧能分开 nonsemantic 和 flat baselines 大量混合的系统效应。R114/R191/R229/R232/R234 说明 scoped exact lineage 在固定 command-mode、target HTTP、controlled multi-workspace、外部仓库和局部 agent-family workloads 上成立; R235–R240 同时暴露 broad raw/Claude target-network 边界, R230–R233 说明 full-history projection 和 duplicate-index normalization 可审计。R160/R200/R220/R248/R253/R254/R256 说明 bounded artifact、public fixture、install/readback 和 crate-package dry-run 已经可审计, 但还不是社区级 artifact 结论。顶会版本必须继续证明 Codex/Claude-launched target-network 边界、更多 agent family 和任意仓库 scope、same-tag prompt-row identity policy、标签 adequacy 和用户任务收益。在当前证据下, 最诚实的论文定位是: 一个有完整系统化方向和真实 characterization 的语义系统效应 profiler, 而不是已经完成的 OSDI artifact。

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] Benjamin H. Sigelman et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report, 2010.
- [3] SigNoz. Understanding Flame Graphs for Visualizing Distributed Tracing. <https://signoz.io/blog/flamegraphs/>.
- [4] SigNoz. How Inkeep Monitors Their AI Agent Framework with SigNoz. <https://signoz.io/blog/inkeep-ai-agent-monitoring/>.
- [5] Datadog. Trace View. https://docs.datadoghq.com/tracing/trace_explorer/trace_view/.
- [6] LangChain. LangSmith tracing documentation. <https://docs.langchain.com/langsmith/>.
- [7] Langfuse. Observability documentation. <https://langfuse.com/docs/observability>.
- [8] Arize Phoenix. Tracing documentation. <https://arize.com/docs/phoenix>.